

Guión de Práctica 4: Memoria Dinámica

Metodología de la Programación
Grado en Ingeniería Informática

Curso 2025/2026

1 Parte 1: Operaciones básicas para manejo de memoria dinámica

1.1 Reservando vector en memoria dinámica con new/delete

Escribir un programa que pida al usuario un número de palabras a introducir, reserve memoria para almacenarlas e imprima el resultado en orden inverso al introducido. El vector debe reservarse de forma dinámica usando la mínima memoria posible.

1.2 Reservando memoria dentro de una función

Implementar una función que, dado un vector de enteros cuyos elementos pueden ir del 0 al 9, devuelva un nuevo vector en el que cada elemento esté repetido tantas veces como indique su valor. Implemente un `main` donde se realice la lectura de datos, se llame a esta función y se muestre el contenido de ambos vectores.

Ejemplos:

- Entrada: $\{1, 3, 2\}$ $\rightarrow$ Salida: $\{1, 3, 3, 3, 2, 2\}$
- Entrada: $\{0, 1, 4, 2\}$ $\rightarrow$ Salida: $\{1, 4, 4, 4, 4, 2, 2\}$

2 El juego de la vida

2.1 Introducción

El Juego de la vida es un autómata celular diseñado por el matemático británico John Horton Conway en 1970. Es un juego de cero jugadores, en el que su evolución es determinada por un estado inicial, sin requerir intervención adicional.

Desde su publicación, ha atraído mucho interés debido a la gran variabilidad de la evolución de los patrones. Se considera que el Juego de la vida es un buen ejemplo de emergencia y autoorganización. Es interesante para científicos, matemáticos, economistas y otros observar cómo patrones complejos pueden provenir de la implementación de reglas muy sencillas.

2.2 Descripción del Problema

El "tablero de juego" es una malla plana formada por cuadrados (las "células") que se extiende por el infinito en todas las direcciones. Por tanto, cada célula tiene 8 células "vecinas", que son las que están próximas a ella, incluidas las diagonales. Las células tienen dos estados: están "vivas" o "muertas" (o "encendidas" y "apagadas"). El estado de las células evoluciona a lo largo de unidades de tiempo discretas (se podría decir que por turnos). El estado de todas las células se tiene en cuenta para calcular el estado de las mismas al turno siguiente. Todas las células se actualizan simultáneamente en cada turno, siguiendo estas reglas (<https://playgameoflife.com/>):

- Nace: Si una célula muerta tiene exactamente 3 células vecinas vivas "nace" (es decir, al turno siguiente estará viva).
- Muere: una célula viva puede morir por uno de 2 casos:
 - Sobrepoblación: si tiene más de tres vecinos alrededor.
 - Aislamiento: si tiene solo un vecino alrededor o ninguno.
- Vive: una célula se mantiene viva si tiene 2 o 3 vecinos a su alrededor.

2.3 Especificación Técnica

Deberá implementar los métodos definidos en la siguiente figura. No se permite modificar el prototipo de los métodos, ni modificar o añadir los datos miembro de la clase.

Listing 1: Archivo conway.h

```
1  #ifndef GAME_OF_LIFE_H
2  #define GAME_OF_LIFE_H
3
4  #include <iostream>
5  #include <fstream>
6
7  class GameOfLife {
8  public:
9      /**
10       * @brief Constructor de la clase GameOfLife.
11       * @param rows Numero de filas de la matriz.
12       * @param cols Numero de columnas de la matriz.
13       */
14     GameOfLife(int rows, int cols);
15
16     /**
17      * @brief Destructor de la clase GameOfLife. Debe invocar a la
18      *        funcion de liberacion.
19      */
20     ~GameOfLife();
21
22     /**
23      * @brief Inicializa el tablero con valores aleatorios.
24      * // Initialize a random number generator
```

```

24     *   #include <random.h>
25     *   random_device rd;
26     *   mt19937 gen(rd());
27     *   uniform_int_distribution<> distrib(min, max);
28     *
29     *   // Generate random number in the range [min, max]
30     *   int randomValue = distrib(gen);
31     */
32 void initializeRandom();
33
34
35 /**
36  * @brief Inicializa el tablero con valores aleatorios.
37  * @param numAliveCells Numero maximo de celdas vivas a
38  *   establecer.
39  */
40 void initializeRandom(int numAliveCells);
41
42 /**
43  * @brief Calcula la siguiente generacion del tablero.
44  * ES NECESARIO USAR UN TABLERO INTERMEDIO CON MEMORIA
45  * DINAMICA.
46  */
47 void nextGeneration();
48
49 /**
50  * @brief Imprime el estado actual del tablero en la consola.
51  */
52 void printBoard() const;
53
54 /**
55  * @brief Guarda el estado actual del tablero en un archivo.
56  * @param filename Nombre del archivo.
57  */
58 void saveToFile(const std::string& filename) const;
59
60 private:
61     int rows, cols;
62     bool** board; ///< Matriz dinamica (vector de punteros).
63
64     /**
65      * @brief Asigna memoria para el tablero. Utilizar el esquema
66      * siguiente
67      * en el que board es un vector de punteros a bool
68      * board -> P -> B B B B
69      *          P -> B B B B
70      *          P -> B B B B
71      *          P -> B B B B
72      *          P -> B B B B
73      */
74 void allocateBoard(); ///< Asigna memoria para el tablero.

```

```

72     /**
73     * @brief Libera la memoria asignada para el tablero.
74     */
75     void deallocateBoard(); ///< Libera la memoria asignada.
76     /**
77     * @brief Cuenta la cantidad de vecinos vivos de una celda
78         específica.
79     * @param row Fila de la celda.
80     * @param col Columna de la celda.
81     * @return Número de vecinos vivos.
82     */
83     int countNeighbors(int row, int col) const;
84 };
#endif

```

2.4 Instrucciones de Implementación

1. **Gestión de Memoria:** El método `allocateBoard` debe reservar memoria para el array de punteros y, posteriormente, para cada una de las filas. El destructor debe invocar a `deallocateBoard` para liberar todos los recursos sin dejar fugas (*memory leaks*).
2. **Cálculo de Generación:** Al implementar `nextGeneration`, se debe usar una matriz auxiliar dinámica. No se deben modificar los valores del tablero original hasta haber calculado el estado completo de la nueva generación.
3. **Visualización:** El método `printBoard` se proporciona implementado en el fichero `conway.cpp`. El método `saveToFile` se proporciona también parcialmente implementado, necesitando solo las instrucciones necesarias para escribir 1 si la célula está viva o 0 en caso contrario.
4. Se proporciona un **programa principal** para probar la funcionalidad desarrollada en el fichero `main.cpp`.
5. Utilizar **compilación separada** con la estructura de carpetas vista en la práctica 1 y entregar fichero `Makefile`.

3 Vector dinámico

Este ejercicio se deberá implementar utilizando **compilación separada**. La definición de la clase y las cabeceras de los métodos se ubicarán en un fichero `.h`, mientras que la implementación de los métodos estará en un `.cpp`. Se incluirá además un tercer fichero con el programa principal.

3.1 Vector Dinámico (Clase VectorSD)

Dada la necesidad de gestionar una estructura de datos de tamaño variable, se propone el diseño de la clase `VectorSD` con los siguientes atributos privados:

- `double *info`: Puntero para la reserva de memoria dinámica.

- `int util`: Número de componentes actualmente usadas.
- `int reservado`: Capacidad total de memoria reservada actualmente.

Se deben implementar los siguientes métodos:

3.1.1 Constructor e Inicialización

El constructor debe inicializar una instancia de la clase reservando n posiciones de memoria dinámica e inicializando el número de componentes usadas a 0.

```
1 VectorSD(int n);
```

3.1.2 Añadir elemento

Método que añade un elemento al final del vector. Si se sobrepasa la reserva de memoria actual (`util == reservado`), el método debe **realojar** el vector reservando el **doblo** de posiciones respecto a la reserva anterior.

```
1 void aniadir(double miInfo);
```

3.1.3 Copiar

Método que devuelve un puntero a un objeto de tipo `VectorSD` que sea una copia exacta del actual. La copia debe optimizar la memoria, reservando espacio **solo para las componentes usadas** (`util`) del vector original.

```
1 VectorSD *copiar() const;
```

3.1.4 Destructor (Liberar memoria)

Implementar el destructor de la clase para asegurar que se libere toda la memoria dinámica reservada por el objeto al finalizar su ciclo de vida.

```
1 ~VectorSD();
```

3.2 Ejemplo de Programa Principal (main.cpp)

El siguiente programa ilustra el uso de la clase `VectorSD`. Se debe completar el programa principal con las instrucciones necesarias para realizar las tareas 4 y 5.

Listing 2: Implementación del main con gestión de punteros

```
1 #include <iostream>
2 #include "VectorSD.h"
3
4 using namespace std;
5
6 int main() {
7     // 1. Inicialización: Capacidad inicial de 2 elementos
8     VectorSD miVector(2);
```

```

9
10 // 2. Añadir elementos: El tercer elemento fuerza redimensión
11 cout << "Añadiendo elementos al vector original..." << endl;
12 miVector.aniadir(1.17);
13 miVector.aniadir(3.40);
14 miVector.aniadir(0.56);
15
16 // 3. Copiar: El método devuelve un PUNTERO a un nuevo objeto
17 cout << "Creando copia exacta en memoria dinámica..." << endl;
18 VectorSD* punteroCopia = miVector.copiar();
19
20 // 4. Mostrar resultados del vector original
21 cout << "\n--- Vector Original ---" << endl;
22
23
24 // 5. Mostrar resultados de la copia a través del puntero
25 cout << "\n--- Vector Copia (vía puntero) ---" << endl;
26
27
28 // 6. Liberación de memoria dinámica
29 // 'miVector' se libera automáticamente (stack), se produce
    una invocación implícita al destructor.
30 // 'punteroCopia' requiere liberación manual con delete (heap)
    , hay que hacer una liberación explícita.
31 delete punteroCopia;
32
33 cout << "\nMemoria de la copia liberada correctamente." <<
    endl;
34
35 return 0;
36 }

```

4 Dependencias de Asignaturas

Se desea gestionar un catálogo de asignaturas donde cada una puede tener múltiples prerequisites. Para optimizar el uso de memoria, **no almacenaremos copias de las asignaturas, sino punteros a las instancias ya existentes**. Se debe utilizar compilación separada para el desarrollo de este ejercicio.

4.1 Definición de la Clase Course

La clase Course gestiona dinámicamente un vector de punteros a otros objetos de tipo Course.

Listing 3: Estructura de la clase Course

```

1 class Course {
2 private:
3     string name;

```

```

4      Course **requisites; // Vector dinamico de punteros a objetos
           Course
5      int num_requisites;
6
7  public:
8      Course(string n) : name(n), requisites(nullptr),
           num_requisites(0) {}
9
10     // Metodos de acceso y modificacion
11     void setRequisites(Course **reqs, int n) {
12         this->requisites = reqs;
13         this->num_requisites = n;
14     }
15     string getName() const { return name; }
16     int getNumRequisites() const { return num_requisites; }
17     Course** getRequisites() const { return requisites; }
18 };

```

4.2 Implementación del método mydeps

El objetivo es implementar una función que extraiga los nombres de las dependencias y los devuelva en un nuevo vector dinámico.

```

1  /**
2   * @brief Devuelve un vector dinamico con los nombres de los
           prerrequisitos.
3   * @param course Referencia al curso a consultar.
4   * @return Puntero a un array de strings (debe ser liberado por el
           usuario).
5   */
6  string* mydeps(const Course &course);
7  }

```

4.3 Inicialización del Sistema

Se debe implementar un programa principal que configure el siguiente esquema de dependencias para 4 asignaturas e invoque a la función `mydeps` para cada una de las asignaturas, mostrando el resultado por pantalla. El programa principal será el responsable de la gestión de toda la memoria dinámica necesaria y de realizar, por tanto, la liberación de la misma.

1. **Asignaturas:** FP, MP, CAL, ED.
2. **FP:** Depende de MP.
3. **ED:** Depende de FP y MP.
4. **CAL:** Sin prerrequisitos.